



UNIVERSITÀ DI PISA

Dipartimento di Informatica
Corso di Laurea Triennale in Informatica

Corso a Libera Scelta - 6 CFU

Interazione Uomo-Macchina

Professori:

Prof. Daniele Mazzei
Tommaso Turchi
Roberto Figliè

Autore:

Filippo Ghirardini

Anno Accademico 2023/2024

Contents

1	Introduzione	3
1.1	Progettazione	3
1.1.1	Design	3
1.1.2	Pensiero computazionale	4
1.1.3	Confronto	4
2	Persone	5
2.1	Human Centered Design	5
2.1.1	Fasi di un processo	5
2.1.2	Activity Centered Design	5
2.2	Progettazione dell'interazione	6
2.2.1	Understanding	6
2.2.2	Discoverability	6
2.3	Perché delle azioni	8
2.3.1	Golfi	8
2.3.2	Stadi delle azioni	8
2.3.3	Fondamenti cognitivi	9
2.4	Personas	11
2.5	Requisiti	11
2.6	User stories	11
2.7	Scenari	12
2.7.1	Scrittura	12
2.8	Casi d'uso	12
2.8.1	Scrittura	13
2.9	Errore umano	13
2.9.1	Analisi della causa	13
2.9.2	Tipologie	14
2.9.3	Prevenzione	14
2.9.4	Mitigazione	15
2.10	User behavior patterns	15
2.11	Information architecture	16
2.11.1	Organization Schemes	16
2.11.2	Organizational Structures	17
2.11.3	Navigation	17
2.11.4	Componenti	17
2.12	Innovazione	18
2.12.1	Design Methods	18
2.12.2	Development methods	20

Interazione Uomo-Macchina

Realizzato da: Ghirardini Filippo

A.A. 2025-2026

1 Introduzione

L'obiettivo del corso è di studiare gli strumenti necessari a comprendere e gestire il processo di sviluppo delle interfacce e dei prodotti interattivi. È fondamentale che l'informatico crei un prodotto **apprezzato** dalle persone.

1.1 Progettazione

1.1.1 Design

Definizione 1.1.1 (Design). *Per design si intende sia il processo di **progettazione** e **pianificazione** sia l'**output** stesso di questo processo.*

Nel design il primo passo è capire **perché** un problema esiste e se ha davvero bisogno di una soluzione. È quindi fondamentale fare in modo che il software risolva i problemi dell'utente, senza aggiungere funzionalità inutili non necessarie all'utente.

Nel design del prodotto è spesso necessario modificare requirement e specifiche per andare in contro alle esigenze degli utenti, rendendo quindi lo sviluppo un processo **antropocentrico** di adattamento del software.

Esistono varie sotto discipline del design, tra cui:

- **Interaction design**: l'attività di progettazione dell'interazione che avviene tra esseri umani e oggetti in generale. Ha come obiettivo quello di rendere macchine, servizi e sistemi **usabili** per gli utenti target, mettendo al centro i loro **bisogni**. Si suddivide in:

- Design di **prodotto**: progettazione di beni e servizi il cui obiettivo principale è quello di essere utilizzati da quanti più utenti possibili migliorandone la vita¹

Definizione 1.1.2 (Designer). *L'inventore o designer del prodotto si focalizza su un problema da risolvere e inventa un prodotto che grazie alle sue caratteristiche tecniche permette di risolvere un problema.*

- Design dell'**esperienza utente** o **UX**: il processo volto ad aumentare la soddisfazione e la fedeltà del cliente migliorando l'**usabilità** e la piacevolezza. Aggiunge alla tradizionale progettazione gli aspetti di business, marketing e sviluppo prodotto.

Definizione 1.1.3 (UX Designer). *Lo UX Designer ha l'obiettivo di far vivere all'utente del suo prodotto la miglior esperienza possibile, evitando di farlo sentire frustrato e deluso.*

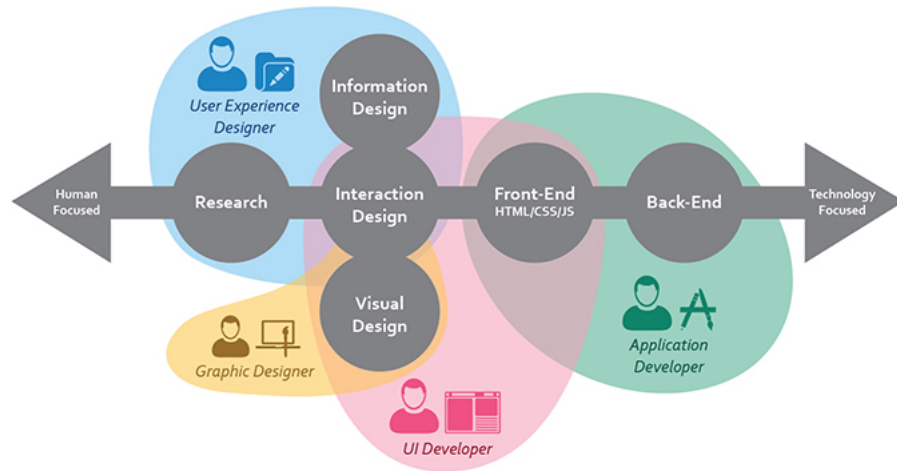
- Design dell'**interfaccia** o **UI**: ha come focus il modo in cui le persone interagiscono con la tecnologia e cerca di migliorare la loro comprensione di ciò che si può fare. Dalla UX si crea uno schema di interazione che serve a progettare l'interfaccia utente al fine di abilitare l'esperienza progettata.

Definizione 1.1.4 (UI Designer). *Lo UI Designer progetta l'aspetto **estetico** e la struttura dell'interfaccia così che questa durante l'utilizzo induca l'utente nel seguire l'esperienza progettata. Produce un **wireframe**² dell'interfaccia e delle linee guida che verranno seguite dagli **UI Developer** per l'implementazione.*

- **Human Machine e Human Computer Interaction**: ha come obiettivo quello di rendere possibile e **facilitare** al massimo per un essere umano l'**uso** e l'**interazione** con i sistemi del mondo IT

¹Spesso usato in modo interscambiabile con **design industriale**: processo strategico di risoluzione dei problemi che guida l'innovazione e porta a una migliore qualità della vita attraverso prodotti, sistemi, servizi ed esperienze innovative

²Bozza grafica



1.1.2 Pensiero computazionale

Nel mondo del *design*, in particolare quello industriale, ci si approccia alla progettazione e risoluzione dei problemi in modo diverso dal mondo informatico.

Per questo motivo nel 2006 Jeannette Wing, direttrice del dipartimento di Informatica della Mellon University, formula la seguente definizione:

Definizione 1.1.5 (Pensiero computazionale). *Il pensiero computazionale è un processo di formulazione di problemi e soluzioni in una forma che sia eseguibile da un agente che processi informazioni.*

Questo consiste non solo nel saper programmare ma anche nel pensare a diversi **livelli di astrazione**, abituando al *rigore* e rendendo possibili gli *atti creativi*. Permette di interagire con persone e strumenti e di usare le macchine come oggetti capaci di compensare le lentezze o l'imprecisione dell'uomo.

1.1.3 Confronto

Se nel **design** la progettazione è affrontata in maniera globale, nel tentativo di comprendere il problema nel suo insieme, nel **pensiero computazionale** si basa sulla suddivisione di un problema in sotto-problemi in modo da giungere ad una formulazione del problema come *algoritmo*.

È importante evidenziare che il design dell'interazione e il pensiero computazionale non sono mutualmente esclusivi ma anzi è nell'**unione** dei due e nella loro **integrazione** che si crea software di qualità.

2 Persone

2.1 Human Centered Design

I progettisti devono produrre cose che soddisfino i bisogni della gente in termini di funzioni, facilità d'uso e gratificazione emotiva, ovvero come un'**esperienza totale**.

L'obiettivo è quello di contrastare la sempre maggiore frustrazione dovuta alla complessità degli oggetti quotidiani: se le macchine hanno una modalità di funzionamento *logica*, gli umani sono l'opposto e di conseguenza nell'interazione tra i due si va a creare una relazione fra specie diverse con modalità di pensiero e funzionamento opposte.

Definizione 2.1.1 (Human Centered Design). *È una **forma di pensiero** per lo sviluppo dei sistemi interattivi che mira a renderli **utilizzabili** e **utili** concentrandosi sugli utenti e i loro bisogni. Questo approccio migliora l'**efficacia**, l'**efficienza**, la **soddisfazione dell'utente**, l'**accessibilità** ed evitando effetti negativi sulla salute, sicurezza e performance.*

Nel design **antropocentrico** si inverte il paradigma di progettazione (*tecno-centrico*) e si mette l'utente e i suoi bisogno al centro del processo, invece delle funzionalità del prodotto. Per farlo è fondamentale **capire le persone** osservandole, dato che spesso loro stessi non sanno di cosa hanno bisogno.

È soprattutto fondamentale la **comunicazione**, specialmente dalla macchina all'utente e in particolare quando qualcosa va storto in modo che l'utente venga guidato nella risoluzione e gestione dell'errore.

L'obiettivo deve essere quello di creare **empatia** nell'utente. I ricordi hanno la capacità di far provare emozioni più profonde rispetto al presente e un utente che ha memoria di un successo favorito dalla tecnologia svilupperà empatia e bisogno per quel prodotto.

Esempio 2.1.1 (Bitcoin). La tecnologia *blockchain* è nata in quanto c'era un bisogno per un metodo alternativo di scambiare moneta (cryptomoneta). In seguito si è provato ad applicare questa nuova tecnologia ad altri ambiti ma senza successo in quanto non si stava creando una soluzione ad un problema ma si stava cercando un problema per una soluzione.

2.1.1 Fasi di un processo

Un processo HCD può essere schematizzato come un flusso continuo ed iterativo che attraversa le seguenti fasi:

- Specificare il **contesto d'uso**: identificare la user base, per cosa utilizzeranno il prodotto e sotto quali condizioni e vincoli
- Specificare i **requirements**: identificare i business requirements e gli obiettivi utente che devono essere raggiunti utilizzando il software
- **Progettare la soluzione**: può essere divisa in più sotto fasi iterative, tipicamente dalle *bozze* ai *prototipi* e alla *soluzione*
- **Testare e valutare**: fondamentale per iniziare di nuovo il ciclo in base ai risultati e procedere quindi ad un miglioramento incrementale

Note 2.1.1. È fondamentale implementare nel software sistemi di **tracking** dell'utente finalizzati alla produzione di statistiche di utilizzo.

2.1.2 Activity Centered Design

È un'estensione di HCD e consiste nel mettere enfasi sulle attività che un utente potrebbe fare con una determinata tecnologia.

I **controlli** Activity Centered (e.g. interruttore per accendere il proiettore) sono eccellenti in teoria ma se realizzati male crea molte difficoltà. Devono quindi essere **User-Activity-Centered** e non Device-Centered

2.2 Progettazione dell'interazione

Il problema principale è che il *buon design* non è universale in quanto un prodotto non può essere apprezzato da tutti perché l'esperienza è soggettiva e dipende dalla persona.

Le macchine sono limitate a seguire **regole fisse e rigide** e si comportano come programmate anche se l'utente si discosta leggermente da esse, nonostante questo possa significare fare qualcosa di totalmente insensato. Questo perché, a differenza degli umani, non hanno il **buon senso** e inoltre spesso le regole da esse seguite sono conosciute solo dalla macchina e dai designers.

Sviluppatori ed ingegneri tendono a compiere l'errore di pensare che la spiegazione logica sia sufficiente per avere un buon design, mentre ovviamente non è così.

Esempio 2.2.1 (Three Mile Island accident). L'incidente al reattore nucleare fu causato da un pessimo sviluppo dell'interfaccia di controllo: nonostante una valvola fosse bloccata in posizione aperta, il led che doveva indicare se fosse chiusa era acceso. Gli operatori impiegarono diverse ore prima di scoprire che indicava solo se il solenoide riceveva corrente e non lo stato di chiusura.

2.2.1 Understanding

Definizione 2.2.1 (Understanding). *È la capacità del prodotto di farsi usare correttamente dall'utente.*

In pratica è la proprietà associata a quanto bene un prodotto dice come si usano le sue funzioni disponibili. È necessario che risponda alle seguenti domande:

- **Come** si usa il prodotto?
- Che **funzione** ha ciascun controllo?
- Come si **combinano** i controlli?

2.2.2 Discoverability

Definizione 2.2.2 (Discoverability). *È la capacità di un sistema di veicolare e comunicare i propri possibili usi all'utente.*

L'idea è quindi di far capire all'utente a cosa serve il prodotto, tipicamente lavorando sulla **visibilità**. Questo non implica che poi lo si sappia usare.

Questa caratteristica si basa su sei principi fondamentali:

- **Affordances**: il termine si riferisce alla **relazione** tra le *proprietà di un oggetto* fisico e le *capacità di un agente* che determina come un oggetto possa essere utilizzato. Il contrario è **anti-affordance**, ovvero la prevenzione dell'interazione.

Esempio 2.2.2. La maggior parte delle sedie possono essere trasportate da una persona (they *afford* lifting). Se una persona debole non riesce a sollevarla, it *doesn't afford* lifting.

Perché questo principio sia efficace è necessario che le affordances e le anti-affordances siano **discoverable** e **perceivable**. Un esempio negativo è il vetro.

- **Signifiers**: specifica **dove** l'azione deve essere eseguita (non quale) e comunicano quindi come usare il design. Possono essere **deliberati** ed **intenzionali**, come il segno "spingi" sulla porta, o **accidentali** e **non intenzionali**, come un percorso nell'erba causato da tante persone che ci hanno camminato.

Per passare da un'affordance a un signifier di solito si procede tramite **convenzioni**, come ad esempio il fatto che una maniglia su una porta indica che devi usarla per aprirla. L'interpretazione di una *perceived affordance* è una **convenzione culturale**.

- **Mapping:** indica la **relazione** tra gli elementi di due insiemi di cose. È importante soprattutto nel design e layout dei controlli e degli schermi in quanto utilizza **analogie spaziali** tra i controlli e i dispositivi controllati per indicare come utilizzarli (e.g. fornelli). Alcuni mappings sono biologici (muovere la mano in alto e in basso indicano più e meno) mentre altri culturali (indicatore del carburante pieno a destra o sinistra).
- **Feedback:** la comunicazione del risultato di un'azione. Deve essere **immediato** ed **informativo** senza ostruire altro (e.g. troppe notifiche ti portano a disattivarle completamente).
- **Conceptual model:** una spiegazione, di solito molto semplificata, di come un qualcosa funziona (e.g. cartelle di file sul pc). Non deve essere completa o accurata, basta che sia utile. È fondamentale che l'**assunzione** su cui si basa la semplificazione rimanga vera (e.g. far vedere i file anche se sono nel cloud, la connessione deve esserci). Sono spesso inferiti dal dispositivo stesso, tramandati da persone, estratti da manuali o costruiti con l'esperienza.
I **mental model** sono invece come le persone rappresentano il funzionamento delle cose nella loro mente. È diverso per ognuno e una persona singola può averne anche diversi per lo stesso oggetto, a volte anche in conflitto.
La combinazione di informazioni necessarie per la creazione di un *conceptual model* di un dispositivo con cui interagiamo è la **system image**. Ad esempio aspetto fisico, somiglianza con altri dispositivi già usati, pubblicità, articoli di giornale o manuali.
Quando la system image è incoerente o inappropriata, l'utente ha difficoltà nell'usare il dispositivo. È importante che il designer trasmetta il suo conceptual model correttamente, tramite la system image, al conceptual model dell'utente.
- **Constraints:** servono a limitare l'insieme delle possibili azioni grazie alla conoscenza del mondo e i modelli concettuali. Possono essere
 - **Fisici**, tra cui le **forcing functions** ovvero casi estremi di constraints che evitano comportamenti inappropriati. Si suddividono in:
 - * **Interlocks:** impone che una serie di operazioni siano eseguite nel giusto ordine
 - * **Lock-ins:** mantiene un'operazione attiva evitando che qualcuno la interrompa prematuramente (e.g. conferma per la chiusura di un programma)
 - * **Lockout:** evita che qualcuno acceda ad uno spazio pericoloso o evita che accada un determinato evento (e.g. limite di età)
 - **Culturali**
 - **Semantici**
 - **Logici**

Note 2.2.1 (Consistenza). Dato che violare delle convenzioni richiede nuovo apprendimento, mantenere una **consistenza** nei vari design è una caratteristica importante.

2.3 Perché delle azioni

È importante analizzare come le persone **sceglono** e **valutano** le loro azioni per costruire un modello concettuale della psicologia umana. Questo implica capire le **emozioni**: **piacere** e **frustrazione** in base al successo delle azioni.

2.3.1 Golfi

Quando le persone usano qualcosa, affrontano due golfi. È compito del *designer* aiutarle a connetterli.

- **Esecuzione**: sforzo mentale per tradurre gli obiettivi in azioni fisiche sul sistema. È correlato alla *discoverability* (l'utente capisce cosa è possibile fare?), alle *affordances* (l'utente interpreta cosa un componente permette di fare?) e alla capacità fisica di eseguire l'azione. I principali elementi del design coinvolti nella connessione sono:
 - *Signifiers*
 - *Constraints*
 - *Mappings*
 - *Conceptual model*
- **Valutazione**: capire se i cambiamenti percepiti hanno avvicinato il sistema all'obiettivo. Richiede un *feedback* continuo sui risultati e sullo stato corrente: l'utente deve sviluppare un senso di *controllo* e comprensione. È importante la **percezione** dell'utente sulle informazioni visuali. I principali elementi del design coinvolti nella connessione sono:
 - *Feedback*
 - *Conceptual model*

2.3.2 Stadi delle azioni

Un'azione si compone dell'**esecuzione** e dell'**interpretazione** dei risultati. Entrambi hanno un effetto sullo stato emotivo dell'utente. Più precisamente dividiamo l'azione in sette stadi:

1. Creazione dell'**obiettivo**
2. **Pianificazione** dell'azione
3. **Specificazione** di un'azione
4. **Esecuzione** dell'azione
5. **Percezione** dello stato del sistema
6. **Interpretazione** della percezione
7. **Confronto** del risultato con l'obiettivo

Le azioni sono **non lineari**: non è necessario attraversare tutti gli stadi, spesso servono più azioni per un singolo obiettivo, possono formarsi sotto-obiettivi e sotto-piani oppure determinati obiettivi possono essere dimenticati, scartati o modificati.

I sette stadi forniscono una linea guida per lo sviluppo di prodotti o servizi; possono essere visti come sette **domande** alle quali chiunque crei un prodotto dovrebbe sempre saper rispondere:

1. Cosa voglio ottenere?
2. Quali sono azioni alternative?
3. Che azione posso fare ora?
4. Come la faccio? Grazie al **feedforward**, che sfrutta *signifiers*, *constraints* e *mappings*
5. Cosa è successo? Grazie al **feedback**, che utilizza le informazioni esplicite sull'impatto dell'azione
6. Cosa significa?
7. Ho raggiunto il mio obiettivo?

Grazie ai sette stadi dell'azione possiamo definire il sette **principi fondamentali del design**:

1. **Discoverability**: è possibile determinare quali azioni sono possibili e lo stato attuale del dispositivo
2. **Feedback**: c'è un flusso continuo e completo di informazioni sul risultato delle azioni e sullo stato del prodotto
3. **Conceptual model**: il design mostra tutte le informazioni necessarie per la creazione di un conceptual model del sistema che permetta la *comprensione* e il *sensu di controllo*
4. **Affordances**: esistono le affordances appropriate per fare le azioni desiderate
5. **Signifiers**: uso efficace dei signifiers per garantire *discoverability* e un *feedback* funzionale e comprensibile
6. **Mappings**: la relazione tra i controlli e le azioni segue il principio di mapping, migliorato quanto possibile attraverso il *layout spaziale* e la *continuità temporale*
7. **Constraints**: fornire constraints di tutti i tipi guida le azioni e facilita l'interpretazione

2.3.3 Fondamenti cognitivi

Innanzitutto possiamo dividere le azioni in due tipi:

- **Opportunistiche**: meno precise ma richiedono meno sforzo mentale, meno problemi e probabilmente più interesse
- **Pianificate**

Inoltre, il pensiero umano si divide ulteriormente in:

- **Conscio**: lento ed elaborato, si analizzano le decisioni, si pensa ad alternative e si valutano diverse opzioni con logica e matematica. Utilizzato durante l'apprendimento, in situazioni di pericolo o quando qualcosa va storto
- **Subconscio**: il cervello confronta la situazione attuale con quelle passate e agisce velocemente e senza sforzo. È ottimo nel riconoscimento di trend e in generale in comportamenti "skilled"

Questo ci porta a definire i due tipi di **memoria**:

- **Dichiarativa**, e.g. ricordare la capitale di un dato paese
- **Procedurale**, e.g. ricordando le attività fatte per arrivare a compiere quell'azione in passato

La cognizione umana è ovviamente **limitata**, sia per *attenzione*, sia *memoria* e che per *ragionamento*. Gli utenti sfruttano aiuti esterni come note o indizi nell'interfaccia: questo implica per il design che il sistema deve compensare per queste limitazioni.

Livelli di elaborazione Un'approssimazione semplicistica di come il cervello elabora le informazioni prevede tre livelli:

- **Viscerale**: comprende i meccanismi di protezione che permettono di fare valutazioni veloci ma semplici sull'ambiente, permettendo di rispondere velocemente e in maniera subconscia. Per i designer questo riguarda la *percezione immediata* (e.g. suono piacevole o meno) e nulla c'entra con l'effettivo funzionamento del prodotto.
- **Comportamentale**: riguarda le abilità apprese che vengono attivate quando si presenta la giusta situazione. Principalmente a livello *subconscio* ma comunque la persona rimane cosciente delle azioni (ma non dei dettagli). Per i designer questo implica che ad ogni azione è associata un'**aspettativa**, positiva o negativa, che porta a *sensu di controllo* o *frustrazione*. È fondamentale il *feedback* per la gestione di questo aspetto.

- **Riflessivo:** riguarda il pensiero *conscio* ed è un processo lento e profondo che avviene dopo che l'evento è successo e comprende le emozioni più complesse. Per i designer è la parte più importante in quanto è qui che si crea un'opinione del prodotto e una memoria di esso.

Riassumendo:

Livello Cognitivo	Fasi Principali	Implicazioni di Design
Viscerale	4. Eseguire 5. Percepire	• Le prime impressioni contano • Attrazione visiva ed estetica • Feedback sensoriale immediato • "Sembra/si sente bene?"
Comportamentale	3. Specificare 6. Interpretare	• Interazioni fluide e abituali • Significanti e mappature chiari • Pattern coerenti • "Posso farlo automaticamente?"
Riflessivo	1. Obiettivo 2. Pianificare 7. Confrontare	• Comprensione più profonda • Modelli concettuali • Memoria e soddisfazione • "Perché è successo? Lo consiglierai?"

Esempio 2.3.1 (Cestino). La metafora fisica del cestino reale con quello digitale. Il **modello mentale** dell'utente è che i file eliminati vanno nel cestino e non sono permanentemente persi finché non si svuota. Funziona perché:

- Riduce il **golfo dell'esecuzione**: gli utenti sanno come eliminare, trascinando i file sul cestino
- Riduce il **golfo di valutazione**: c'è un feedback visivo, l'icona del cestino si riempie e i file sono visibili al suo interno
- Corrisponde all'esperienza nel mondo reale
- Fornisce sicurezza in quanto azione reversibile

Esempio 2.3.2 (Errori senza contesto). Il classico errore di Windows di una stampa fallita è un pessimo esempio in quanto:

- Aumenta il **golfo dell'esecuzione**: l'utente non sa cosa fare dopo, può solo cliccare "OK" per ignorarlo
- Aumenta il **golfo di valutazione**: il messaggio di errore è vago e non specifica quale sia il vero problema se non con un codice di errore che per l'utente non ha significato
- "Nascondi dettagli" suggerisce che ci siano più informazioni, ma in realtà è solo nascosto di default
- Non si crea un **modello concettuale** in quanto l'utente non capisce che cosa sia andato storto

2.4 Personas

La *user persona* è il personaggio che rappresenta un potenziale utente del prodotto e servono ad aiutare il team di design nel loro lavoro, analizzando e dividendo gli utenti per i loro **obiettivi** e le loro **frustrazioni**. La creazione di una *persona* parte dalle analisi degli utenti tramite tecniche, come:

- **Analisi dei task**, e.g. ordinamento di carte
- **Feedback**, e.g. interviste e focus groups
item **Prototyping**, ovvero sperimentare con le idee prima di svilupparle

Questo porta alla definizione di tre tipi di *personas*:

- **Proto**: una versione leggera creata apposta in poco tempo dai designers
- **Qualitative**: create da ricerche esplorative solide (e.g. interviste) con un campione da cinque a trenta utenti, poi segmentati
- **Statistical**: raccolta di dati tramite un sondaggio su un grande numero di utenti per poi cercare *cluster* di risposte analoghe. In realtà funzionano in combinazione con la *qualitative research*

Note 2.4.1 (Pareto principle). Quante persone considerare per la ricerca? Per tanti eventi, circa l'80% degli effetti deriva dal 20% delle cause.

All'interno di ogni *persona* troviamo diverse informazioni:

- Informazioni **demografiche**, e.g. età e salario
- **Dettagli personali**, e.g. biografia e fotografie
- **Comportamento** e/o dettagli cognitivi, come il modello mentale e come si sente riguardo alle attività da compiere
- **Obiettivi e motivazioni** per l'utilizzo del prodotto
- Dettagli **comportamentali** su come la persona si comporta durante l'utilizzo del prodotto

Note 2.4.2 (Archetype). La *persona* è quella fisica mentre l'astrazione è l'**archetipo**.

2.5 Requisiti

Definizione 2.5.1 (Requisito). *Un requisito è un servizio, funzione o caratteristica di cui un utente ha bisogno.*

È importante specificare che i requisiti sono dinamici tanto quanto l'ambiente in cui nascono. Possiamo comunque dividerli in due categorie:

- **Funzionali**: esprimono una funzione o caratteristiche e definiscono cosa è necessario, e.g. visitare il sito del cliente. **Non** includono **come** implementare fisicamente la soluzione.
- **Non funzionali**: definiscono quanto bene o a che livello una soluzione si trova, tramite caratteristiche come sicurezza, manutenibilità, disponibilità, etc. . . Possono essere relativi ad un gruppo di *requisiti funzionali* o ad un singolo

2.6 User stories

Definizione 2.6.1 (User story). *È una breve frase o riassunto che identifica l'utente e il loro obiettivo/necessità dal loro punto di vista.*

Le *user stories* aiutano a descrivere le informazioni pratiche sugli utenti, e.g. necessità e motivazioni per un servizio, e aiutano il team di sviluppo a creare una *roadmap*.

As a [role], I want [feature] because [reason].

Possono essere scritte da chiunque, ma è necessario precisare che non serve che sia il proprietario del prodotto a farlo. In un progetto con sviluppo agile, ogni componente del gruppo dovrebbe scriverle. In ogni caso è più importante il protagonista della *story* rispetto all'autore. Possono essere scritte in diversi livelli di dettaglio, anche se tendenzialmente quelle più grandi vengono divise in molteplici piccole.

2.7 Scenari

Definizione 2.7.1 (Scenario). *Uno scenario è una situazione che descrive come gli utenti eseguono azioni sul tuo sito o applicazione.*

Gli scenari sono in pratica lo sviluppo della **user story** aggiungendo l'**interazione** con il prodotto o il servizio. Possono essere divisi in più casi d'uso.

Esempio 2.7.1. Uno scenario in cui John usa un'applicazione per il telefono per comprare un biglietto per un workshop di design mentre va a lavoro.

Gli scenari aiutano gli stakeholders a visualizzare l'idea del team di design, fornendo un contesto per come dovrebbe essere l'esperienza dell'utente, permettendo la comunicazione tra il pensiero *creativo* e quello di *business*. Aiutano anche il team di design permettendogli di immaginare la soluzione ideale per il problema di un utente e su quali parti della UX concentrarsi.

2.7.1 Scrittura

Un buon scenario deve essere **conciso** e rispondere alle seguenti domande:

- **Chi** è l'utente? Usando le *personas* sviluppate
- **Perché** l'utente usa il prodotto? Quali sono le sue motivazioni e aspettative?
- Quali **obiettivi** ha? Tramite l'*analisi dei task* capire come soddisfarli
- **Come** fa l'utente a raggiungere gli obiettivi? Quali ostacoli e possibilità ci sono? Non è sempre presente

La pianificazione parte con il **mapping**: l'utente principale viene definito tramite sviluppo di *persona* e l'**attività chiave** è quella che l'utente spera di completare. Il passo successivo è fare un'*analisi* dello scenario, mettendo gli obiettivi dell'utente nel contesto e vedere quali passi dovrebbe fare.

Note 2.7.1. È fondamentale concentrarsi sugli obiettivi dell'utente: ciò che vogliono ottenere dal servizio, il loro contesto, la loro conoscenza pregressa e il loro background.

2.8 Casi d'uso

Un caso d'uso è una **descrizione scritta** di come un utente eseguirà l'azione nell'applicazione e specifica, dal suo punto di vista, il comportamento del sistema. Ogni caso d'uso è rappresentato come una sequenza di passi a partire dall'obiettivo e fino al suo raggiungimento.

La differenza principale dallo *scenario* è la **prospettiva**: il caso d'uso è più granulare e descrive il **flusso** di uno specifico attore. Il caso d'uso aggiunge valore spiegando **come** il sistema si dovrebbe comportare e aiutando a capire cosa potrebbe andare storto. Fornisce una lista di obiettivi che può essere utilizzata per stabilire il costo e la complessità del sistema.

Incluso	NON incluso
• Chi sta usando il sito web • Cosa vuole fare l'utente • L'obiettivo dell'utente • I passaggi che l'utente compie per completare un compito specifico • Come il sito web dovrebbe rispondere a un'azione	• Linguaggio specifico per l'implementazione • Dettagli sulle interfacce utente o sulle schermate

In particolare un caso d'uso si compone dei seguenti **elementi**:

- **Attore**: qualcuno o qualcosa che usa il sistema con un certo comportamento
- **Stakeholder**: qualcuno o qualcosa che ha interesse nel comportamento del sistema in discussione (SUD)
- **Attore principia**: stakeholder che inizia un'interazione con il sistema per raggiungere un obiettivo
- **Precondizioni**: cosa deve essere vero prima e dopo l'esecuzione del caso d'uso
- **Triggers**: evento che causa il caso d'uso
- **Basic flow**: caso d'uso dove va tutto bene
- **Alternative flow**: percorsi alternativi che accadono quando qualcosa va storto nel sistema

2.8.1 Scrittura

Kenworthy (1997) specifica i seguenti passaggi per scrivere un caso d'uso:

1. Identificare chi userà il sito web
2. Scegliere uno di quegli utenti
3. Definire cosa l'utente vuole fare sul sito. Ogni azione fatta diventa un caso d'uso
4. Per ogni caso d'uso, decidere cosa dovrebbe normalmente succedere
5. Descrivere il corso degli eventi in termini di cosa l'utente fa e come il sistema si comporta nei suoi confronti
6. Considerare corsi degli eventi alternativi
7. Trovare similarità tra i vari casi d'uso, estrarle e usarle per creare un caso d'uso comune
8. Ripetere da 2 a 7 per tutti gli altri utenti

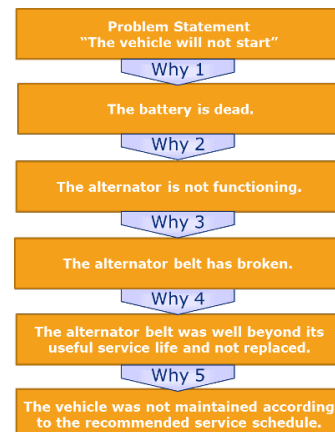
2.9 Errore umano

La maggior parte degli incidenti industriali sono causati da errori umani (c.a. 75% – 95%) e la colpa è del design. Questo perché progettiamo macchinari che richiedono attenzione e allerta costante per ore o procedure arcaiche e confusionarie che vengono usate una volta nella vita. Le persone si ritrovano quindi senza nulla da fare per ore con all'improvviso una situazione in cui bisogna rispondere velocemente e precisamente oppure situazioni in cui sono costantemente **interrotte** mentre devono compiere più attività contemporaneamente.

In generale l'essere umano non è bravo nella precisione: eccelle in creatività, curiosità ed esplorazione. Tutte cose che vanno contro il funzionamento delle macchine.

2.9.1 Analisi della causa

Quando si presenta un errore è fondamentale capire **perché** sia successo. Una buona tecnica è quella sviluppata da Sakichi Toyoda, i "Five Whys", ovvero continuare a chiedere per cinque volte perché partendo dal problema iniziale. In pratica non è detto che ne servano o bastino cinque.



2.9.2 Tipologie

Definizione 2.9.1 (Errori). *L'errore umano è definito come un qualunque discostamento dal comportamento "appropriato" (che spesso è determinato a fatto avvenuto). È il termine generale per tutte le azioni sbagliate.*

Esistono due classi principali di errori:

- **Slips** o lapsus: avviene quando una persona vorrebbe fare un'azione ma finisce per farne un'altra. Di solito fatti dai *neofiti*. Si divide in:
 - **Action-based**: l'azione sbagliata viene eseguita, e.g. verso il latte nel caffè e poi metto l'ultimo in frigo invece del primo
 - **Memory lapses**: la memoria fallisce e l'azione non viene eseguita o risulta non valutata, e.g. dimentico di spegnere il fornello
- **Mistakes** o errori cognitivi: avviene quando viene stabilito l'obiettivo sbagliato o viene formato il piano sbagliato. Da quel momento, anche se le azioni sono eseguite correttamente, fanno tutte parte dell'errore. È quindi il **piano** ad essere sbagliato. Di solito fatti dagli *esperti*. Si divide in:
 - **Rule-based**: la persona ha capito la situazione ma poi decide in base ad un corso di eventi errato, viene seguita la regola sbagliata. E.g. un meccanico trova un problema alla batteria ma decide di non rimpiazzarla perché funziona alla metà delle prestazioni
 - **Knowledge-based**: il problema non viene diagnosticato per conoscenza errata o incompleta, e.g. il peso del carburante è stato calcolato in libbre invece che chili
 - **Memory-lapse**: c'è una dimenticanza nelle fasi di definizione degli obiettivi, piani o valutazione. E.g. un meccanico non trova il problema perché dimentica un passaggio durante un controllo

Interruzioni Le interruzioni sono una delle cause principali degli errori, specialmente i *memory-lapse*. Quando un'attività è interrotta da un altro evento, non solo c'è il tempo sprecato per risolverne la causa ma c'è anche quello necessario a riprendere l'attività originale.

Feedback errati Allarmi non necessari o fastidiosi spingono l'utente a disattivarli. Questo poi porta problemi se si vogliono ripristinare (bisogna ricordarsene) o se avviene un incidente mentre sono disattivati. Un altro problema sono i *feedback parlati*: possono essere molto precisi sul messaggio ma allo stesso tempo confusionari se in ambiente rumoroso o se sovrapposti.

2.9.3 Prevenzione

L'ideale è che un singolo errore non dovrebbe poter fare danni estesi. Alcune buone pratiche:

- **Capire** le cause dell'errore e progettare di conseguenza
- Fare **sensibility check**, ovvero controllare se l'azione rispetta il senso comune. E.g. la banca mi chiede se sono sicuro di fare un bonifico grosso
- Rendere le azioni **reversibili** o rendere difficili le azioni irreversibili
- Rendere gli errori **facili da trovare** e da correggere
- Non trattare l'azione come un errore, piuttosto aiutare la persona a completarla in maniera appropriata
- Aggiungere **constraints**, ovvero blocchi specifici per la sezione dell'azione che potrebbe causare errore. Si dividono in:
 - **Segregate control**: spargere i controlli che possono essere confusi lontani tra loro
 - **Separate modules**: i controlli non rilevanti per l'operazione corrente non sono visibili

Spesso vengono aggiunti messaggi di **richiesta di conferma** per un'azione. Questi sono però inefficaci in quanto di solito la persona ha già deciso di volerla compiere nel momento in cui preme la prima volta il pulsante. È comunque utile metterli, rendendoli più evidenti (e.g. dimensione e colore).

Note 2.9.1 (Minimizzazione degli slip). Dato che il comportamento *skilled* è **subconscio**, per ridurre gli slip non bisogna fare in modo che sia richiesta più attenzione ma piuttosto bisogna assicurarsi che le azioni e i loro controlli siano il più diversi possibili (o fisicamente distanti).

2.9.4 Mitigazione

Per mitigare gli errori bisogna aumentare il numero di controlli e ridurre il numero di punti critici in cui un errore può avvenire. I punti principali per il *design* che tiene conto degli errori sono:

- **Diffondere la conoscenza** richiesta per usare una determinata tecnologia, facendo in modo che chi la conosce bene possa lavorare efficientemente e chi non la conosce può comunque impararla
- Usare **constraints** naturali e artificiali
- Connettere correttamente i due **golfi**

2.10 User behavior patterns

Quando gli utenti interagiscono con le interfacce, il loro comportamento segue dei **pattern** che possono essere predetti. Capirli ci permette di progettare interfacce che sembrano naturali all'utente, supportando strategie e abitudini comuni e creando interazioni più intuitive.

Self Exploration Probabile che impari e si senta bene nel farlo, cliccare finestre pop-up, reinserire dati eliminati per sbaglio o mettere video in pausa. Far sì che sia possibile **esplorare** senza che questo costi per l'utente.

Instant Gratification Queste persone vogliono vedere immediatamente i risultati delle loro azioni e si sentono più fiduciosi in sé stessi. Dato che ogni esperienza positiva porta gratificazione, bisognerebbe progettare la UI in modo che la prima azione sia estremamente semplice. Le **funzionalità introduttive** non dovrebbero essere nascoste.

Satisficing Queste persone scelgono la prima cosa anche se potrebbe essere sbagliata e accettano una risposta "**sufficientemente** buona" invece della migliore se questo fa risparmiare soldi e tempo. Usare etichette brevi, semplici e veloci da leggere in modo che l'utente capisca alla prima lettura. Fornire **metodi di fuga**. Questo comportamento è spesso trovato in utenti inesperti che si trovano ad usare cose complesse.

Changes in Midstream Fornire l'opportunità alle persone di cambiare i propri obiettivi in corso d'opera, a meno che non sia possibile (wizard, hub and spoke, modal panel). Permettere all'utente di iniziare un processo, interromperlo e poi continuarlo successivamente: **reentrance**.

Deferred Choices Utenti che spesso saltano domanda necessarie, rispondendo solo al minimo necessario (per necessità o scelta), per poi tornare più tardi. Mostrare la lista breve e nascondere quella lunga, fornire valori di default e permettere di terminare successivamente.

Incremental Construction Le persone di solito non creano tutto in una volta ma partono da piccoli pezzi, ci lavorano, li testano e vanno avanti. Le interfacce devono essere **responsive** a modifiche e salvataggi veloci. I **feedback** devono essere costanti. L'obiettivo è indurre un "flow" nell'utente, in modo che il godersi l'attività sia esso stesso il premio.

Habituation L'utente non deve più ragionare per compiere azioni abituali, migliorando l'efficienza. Questo però può portare a trappole per l'utente se una *gesture* non funziona o fa qualcosa di distruttivo.

Microbreaks L'utente fa piccole pause tra attività di concentrazione. Queste sono fondamentali per i processi cognitivi e per rilassarsi dallo stress. Le interfacce devono supportare un'interruzione e ripresa delicata e piacevole.

Spatial Memory Spesso le persone trovano gli oggetti ricordandosi dove sono e non come sono chiamati. Posizionare i controlli in posti prevedibili, fornire spazi adattabili dall'utente. L'inizio e la fine delle liste e dei menù sono posti speciali da un punto di vista cognitivo.

Prospective Memory Dare all'utente strumenti per creare i propri metodi di promemoria (e.g. lasciare le cose davanti alla porta per prenderle il giorno dopo). Lasciare artefatti in giro che permettano di identificare attività incomplete.

Streamlined Repetition Gli utenti a volte devono fare la stessa azione ripetutamente: gli va resa facile per evitare di sprecare tempo.

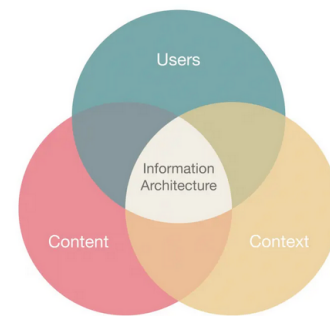
Keyboard Only Collegato a *habituation* e *flow*, rende più facili attività ripetitive fornendo comandi da tastiera che sono più veloci del mouse.

Other People's Advice Le persone influenzate da consigli ed esperienze di colleghi, che siano questi diretti o indiretti. È importante la presenza di una **community**.

2.11 Information architecture

La IA si concentra sull'**organizzare**, **strutturare** ed **etichettare** il contenuto in maniera efficace e sostenibile per aiutare l'utente a trovare le informazioni e completare le attività. Per creare dei sistemi di informazione è quindi necessario capire la natura di:

- **Users:** l'audience, le attività, i bisogni
- **Context:** obiettivi di business, investimenti, politica, cultura, tecnologia, risorse, limitazioni
- **Content:** obiettivi di contenuto, tipi di documenti e dati, volume, strutture preesistenti, comando e proprietà



I componenti principali della IA sono:

- **Organization Schemes and Structures:** come categorizzare e strutturare le informazioni
- **Labeling Systems:** come rappresentare le informazioni
- **Navigation Systems:** come gli utenti esplorano o si muovono all'interno delle informazioni
- **Search Systems:** come gli utenti cercano le informazioni

2.11.1 Organization Schemes

Descrivono come **categorizzare** il contenuto e i diversi modi in cui creare **relazioni** tra i vari componenti. Gli schemi si dividono in:

- **Exact:** dividono le informazioni oggettivamente in categorie mutualmente esclusive. Sono utili per gli Information Architects ma possono essere difficili per gli utenti. Alcuni esempi sono:
 - **Alfabetici:** è importante che le etichette corrispondano a ciò che l'utente cerca
 - **Cronologici:** fondamentale che ci sia un accordo su come datare l'informazione
 - **Geografici**
- **Subjective:** categorizzano le informazione in un modo che può essere specificato o definito dall'organizzazione o dal campo. Sono più difficili da progettare ma spesso più utili degli *exact*. Alcuni esempi:
 - **Argomento**

- **Task:** considera le necessità, azioni, domande o processi che l'utente può avere per quel contenuto
- **Audience:** organizzare i contenuti in base al segmento di utenza. Possono essere *chiusi* o *aperti* se ci si può spostare o meno. E.g. parental control
- **Metafore:** aiuta gli utenti associando i contenuti a concetti familiari, e.g. icone

Note 2.11.1 (Ibridi). È possibile mischiare i diversi tipi di schemi ma è sconsigliato in quanto potrebbe causare confusione agli utenti. Di solito si fa quando il team non riesce a giungere ad un accordo su quale usare.

2.11.2 Organizational Structures

Descrive come è definita la **relazione** tra diversi pezzi del contenuto e permette all'utente di predire dove troverà le informazioni. Le quattro principali strutture sono:

- **Gerarchico:** spesso chiamata ad albero o hub-and-spoke, utilizza un approccio padre-figlio tra i diversi pezzi di informazione. L'utente parte da categorie generali e scende in quelle sempre più dettagliate
- **Sequenziali:** richiedono che l'utente proceda passo passo seguendo un percorso specifico attraverso il contenuto, e.g. processo di acquisto online
- **Matrici:** permettono all'utente di scegliere il loro percorso dato che il contenuto è collegato in molti modi, e.g. la struttura del web
- **Database:** utilizza un approccio dal basso all'alto dove il contenuto dipende fortemente dai collegamenti tra i suoi metadati. Facilita un'esperienza dinamica e permette filtri avanzati e possibilità di ricerca

È fondamentale che gli architetti tengano presente che la struttura scelta deve tollerare miglioramenti e futuri aggiornamenti, in particolare:

- Permettere la **crescita**: si deve poter aggiungere nuovo contenuto e nuove sezioni
- Evitare strutture troppo o troppo poco **profonde**: nel primo caso servono menù giganteschi mentre nel secondo l'informazione è sepolta sotto troppi livelli

2.11.3 Navigation

La navigazione riguarda come l'utente si **orienta** all'interno dell'interfaccia e mira a garantire che l'utente sia capace di trovare l'informazione o la funzionalità che sta cercando. Si basa su due principi fondamentali:

- **Findability:** essere capaci di localizzare l'informazione che l'utente considera rilevante (e.g. biblioteca). È importante piazzare gli elementi con consistenza nell'interfaccia
- **Discoverability:** la possibilità che gli utenti hanno di scoprire nuove informazioni o funzionalità di cui non erano a conoscenza (e.g. libro sconosciuto in biblioteca). Si possono piazzare ad esempio i nuovi contenuti vicino a quelli più popolari. Di solito però l'interfaccia fornisce strumenti che permettono di scoprire nuovi contenuti in maniera indiretta, per poi rendersene conto solo dopo.

2.11.4 Componenti

Durante la progettazione dell'interfaccia è importante rimanere **consistenti** e **prevedibili** per la scelta degli elementi. Anche se gli utenti non lo sanno, hanno sviluppato una familiarità per come gli elementi si comportano e bisogna quindi scegliere di conseguenza, mirando all'efficienza e alla soddisfazione. Alcuni esempi sono:

- **Input Controls:** e.g. checkboxes, radio buttons, dropdown lists, list boxes, bottoni, toggles, campi di testo, date field

- **Navigational Components:** e.g. breadcrumbs, slider, barra di ricerca, paginazione, tags e icone
- **Informational components:** e.g. tooltips, icone, barre di caricamento, notifiche e messaggi
- **Containers:** fisarmonica

2.12 Innovazione

L'innovazione è comunemente definita come l'introduzione di nuove combinazioni, come beni, metodi produttivi, nuovi mercati, etc. . . Per quanto esistano definizioni diverse, in tutte dominano la **novità**, il **miglioramento** e la **diffusione**. Possiamo dire quindi che è un qualcosa di **originale** e **più efficace** che irrompe nel mercato o nella società.

Note 2.12.1 (Invenzione). L'innovazione è correlata ma diversa dall'invenzione: la prima è più diretta all'implementazione pratica della seconda per avere un impatto su un mercato o società. Non tutte le innovazioni hanno bisogno di una nuova invenzione.

Innovazione sostenuta La maggior parte dei processi di innovazione si basano su miglioramenti **incrementali** di prodotti già esistenti. Avvengono un passo alla volta, senza bisogno di cambiamenti nella compagnia, nelle abilità necessarie e rimanendo lenti e a basso rischio, mantenendo il target e il mercato stabile.

Innovazione dirompente È un'innovazione che crea un nuovo mercato e catena di valore, scuotendo quelli esistenti e spiazzando le compagnie, prodotti e alleanze già esistenti. Ad oggi indica avanzamenti tecnologici e scientifici dirompenti. Non tutte le innovazioni sono dirompenti, anche se rivoluzionarie, e.g. la macchina.

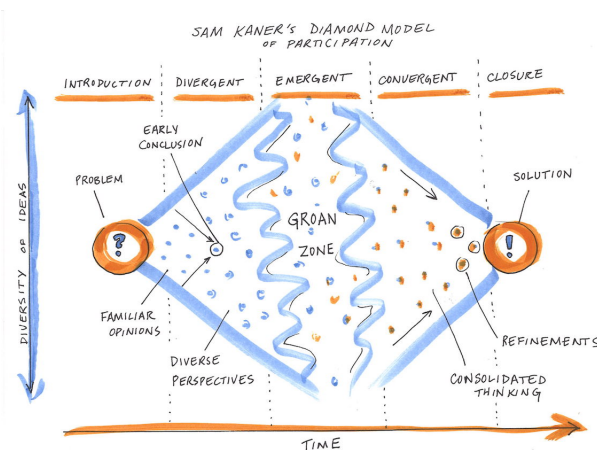
È importante anche distinguere il product **management** dal product **development**: il primo riguarda il **cosa** mentre il secondo il **come**. Il team di sviluppo riceve i requisiti specificati dal manager e li usa per creare un prodotto funzionante che rispecchi gli standard della compagnia. Consiste quindi nel tradurre un concetto in qualcosa di vendibile sul mercato.

2.12.1 Design Methods

Human Centered Design Process L'azienda **IDEO** è una delle più innovative del mondo. Il loro punto chiave è l'empatia per l'utente finale. Credono che la chiave per capire cosa gli umani vogliono veramente risieda in:

- **Osservarne il comportamento:** provare a capire le persone osservandole
- Mettersi nella **stessa situazione** dell'utente finale, per capire come gli utenti si sentono

IDEO definisce HCD Process come un approccio creativo al problem-solving che parte con le persone e finisce con soluzioni fatte apposta per soddisfare le loro necessità.



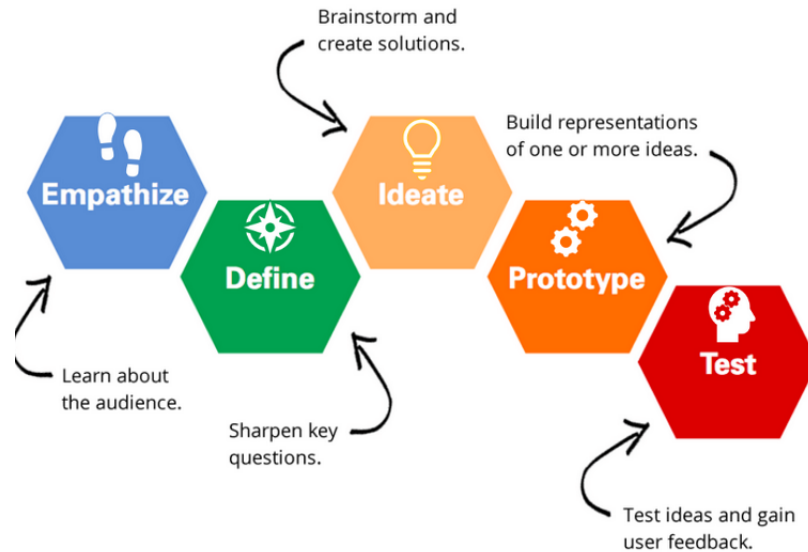
Il processo HCD si divide in tre fasi:

1. **Inspiration:** capire i bisogni degli utenti e le loro difficoltà, dimenticando i preconcetti che si hanno. Bisogna essere aperti a tutte le possibilità.
2. **Ideation:** dopo aver consolidato le ricerche, bisogna visualizzare e discutere tutte le possibili soluzioni, anche se assurde. È fondamentale ricevere feedback e ripartire partendo da quelli in modo da ottenere idee buone. Non serve creare prototipi ma è sufficiente fare sketch o modellini.
3. **Implementation:** in questa fase si crea un prototipo che possa essere provato dagli utenti e la produzione dell'oggetto stesso. Si inizia anche a creare un modello di business.

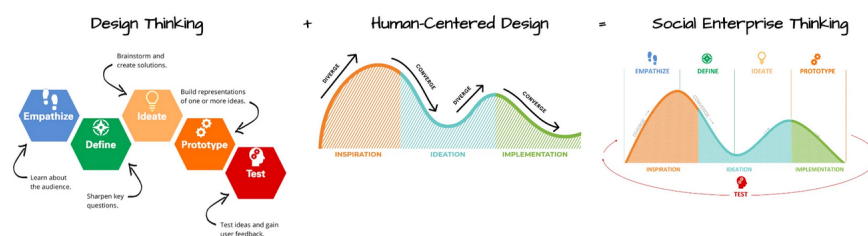
Design Thinking Il Design Thinking è un processo **iterativo** dove si cerca di capire l'utente, mettere in discussione le assunzioni e ridefinire i problemi nel tentativo di trovare strategie e soluzioni alternative che potrebbero non essere ovvie. Allo stesso tempo fornisce un approccio **solution-based** per risolvere i problemi.

È estremamente utile per quei problemi che sono mal definiti o sconosciuti, permettendo di ridefinirli con un approccio **antropocentrico** tramite sessioni di **brainstorming** e approcci **hands-on** per quanto riguarda prototipi e test. Si divide in cinque fasi:

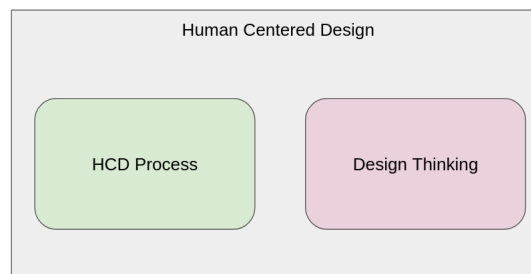
1. **Empatizzare:** studiare i propri utenti
2. **Definire** i bisogni degli utenti, i problemi e le tue intuizioni
3. **Ideare** sfidando le assunzioni e creando idee per soluzioni innovative
4. **Prototipo**
5. **Test**



La combinazione di questi metodi di design porta al **Social Enterprise Thinking**.



Note 2.12.2 (HCD). Applicare lo Human-Centered Design sopra al Design Thinking permette di costruire una soluzione che sia capace di fare soldi e che allo stesso tempo soddisfi le necessità delle persone che la utilizzano. È importante notare che HCD è un **mindset** mentre HCD Process e Design Thinking sono **design methods**.



2.12.2 Development methods

Waterfall È una metodologia di sviluppo lineare, dove i requisiti dei clienti e degli stakeholders sono raccolti all'inizio del progetto e poi viene creato un piano sequenziale per soddisfarli.

Agile È un termine ombrello per un insieme di framework e pratiche basate sui valori e principi espressi nel "Manifesto for Agile Software Development". La differenza principale dagli altri approcci è il concentrarsi sulle **persone** che lavorano e su **come lavorano assieme**. È il metodo migliore per HCD e Design thinking dove iterazioni continue sono fondamentali.

Scrum È un framework agile progettato per piccoli gruppi che dividono il loro lavoro in obiettivi che possono essere completati in breve tempo (circa due settimane), chiamati **sprint**. Il team controlla i progressi in riunioni giornaliere di quindici minuti, chiamati **daily scrums**. Alla fine dello sprint si fa una **review** per mostrare il lavoro fatto e migliorare per i prossimi. Uno dei principi fondamentali è che i clienti **cambieranno idea** su ciò che vogliono o di cui hanno bisogno e ci saranno quindi sfide imprevedibili per cui un approccio pianificato non funzionerebbe. Scrum usa quindi un approccio empirico basato su prove, accettando che il problema non può essere compreso e definito completamente subito all'inizio del progetto. Si concentra piuttosto sull'abilità di rispondere velocemente a nuovi requisiti e di adattarsi a nuove tecnologie e cambiamento del mercato. Si compone di tre ruoli:

- **Product owner**, rappresenta gli stakeholder e i clienti ed è responsabile per consegnare dei buoni risultati di business. Di solito definisce il prodotto in termini "customer-centric" e li aggiunge al **product backlog** in ordine di importanza e dipendenze
- **Development team**
- **Scrum master**: non il classico capo ma agisce da buffer tra il team e ogni distrazione, facendo in modo che il framework sia seguito